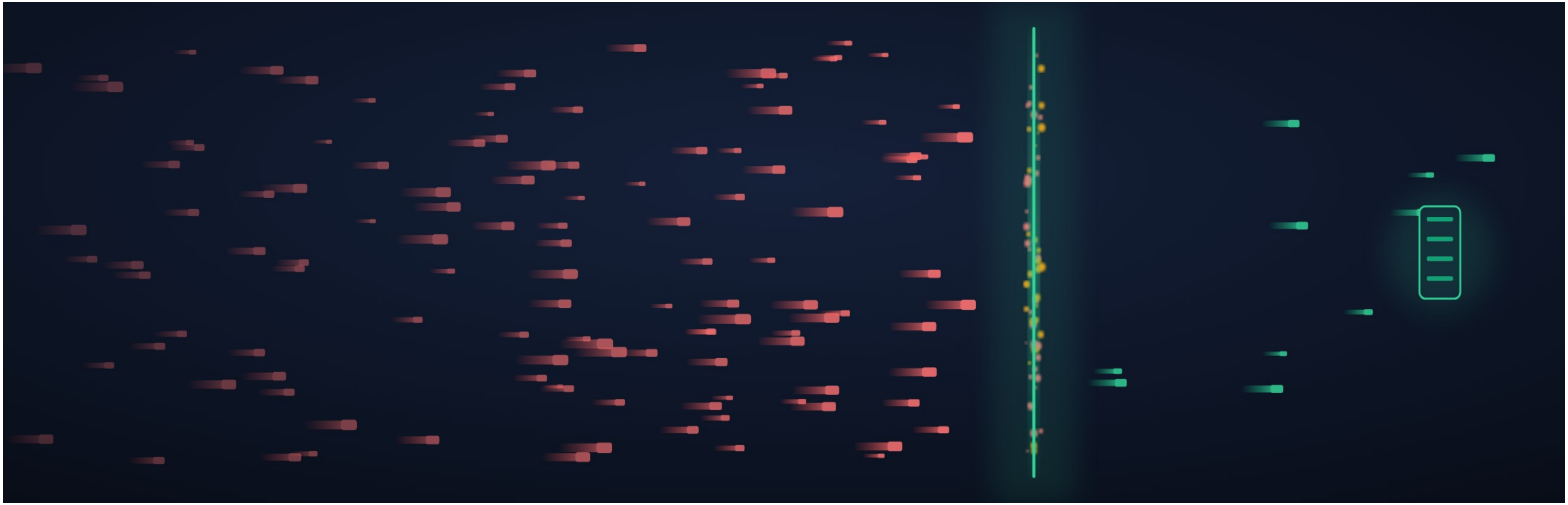


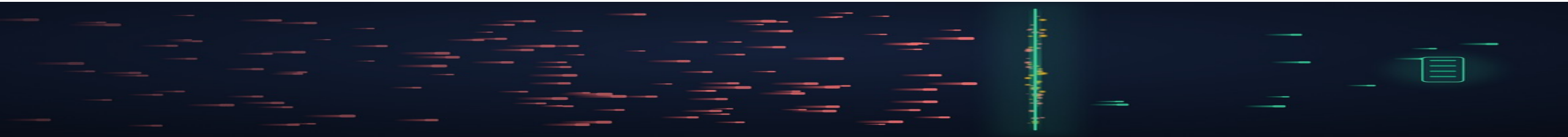
Team Segfault · A3-PS022-TC005

Dropping DNS floods at kernel speed

Meet Segfault

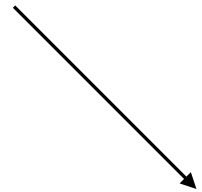
An in-kernel DNS flood defense built on eBPF / XDP.

- Trust by proof, not by rate
- Cost, not count — each query priced by what it costs the server
- Adaptive controller: CALM → GUARD → SIEGE
- Line-rate on server NICs



100%

goodput held in every attack scenario — while every rate-only baseline dropped the real client.



Rate is the lever the attacker controls.



Trust by proof

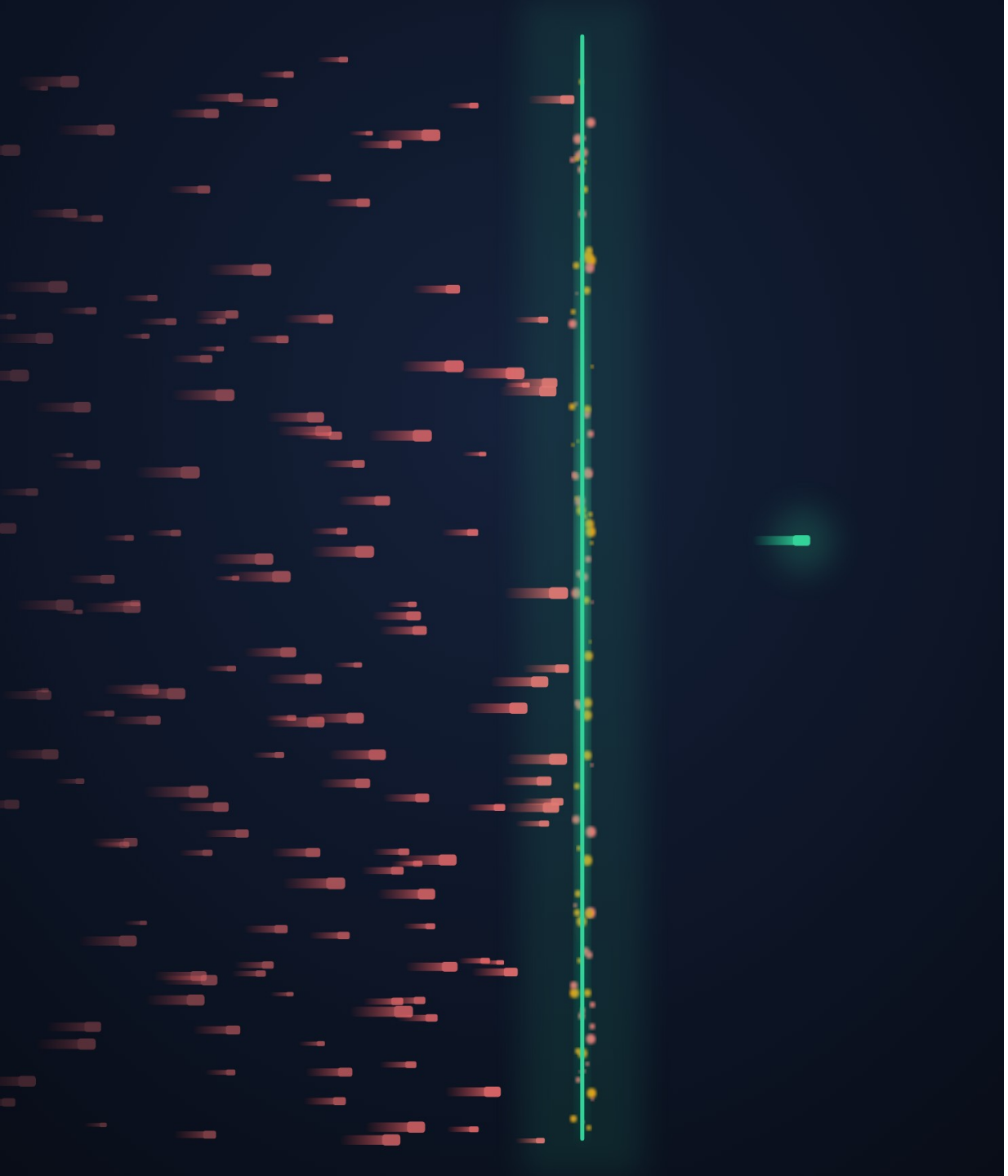
Who, not how fast

Every source earns a trust tier from evidence of a real return path. A spoofer cannot fake a cookie or a completed TCP exchange.

Cost, not count

Each query is priced by what it costs the server, so cheap-to-send, expensive-to-answer floods are shed at the NIC.





How does it hold?

The admission decision

Per-source / per-prefix / per-tier GCRA token bucket

$$t + \tau \geq \mathrm{TAT}$$

$$\mathrm{TAT} \leftarrow \max(\mathrm{TAT},\, t) \;+\; \mathrm{cost}(q) \cdot I$$

Cost-weighted admission

$$\mathrm{cost}(q) = w_{\mathrm{type}} \cdot w_{\mathrm{zone}}(q)$$

A query is admitted only when its bucket has room (t = now, τ = burst). The bucket fills faster for costlier queries — random-subdomain / NXDOMAIN zones are repriced up.

Shedding the unknown, cheaply

Count-Min sketch — no per-source state for tier T0

$$\hat{c}(x) = \min_{1 \le j \le d} C[\, j,\ h_j(x)\,]$$

$$\hat{c}(x)\ \le\ c(x) + \varepsilon N \qquad (\text{prob } 1-\delta)$$

$$w = \lceil e/\varepsilon \rceil, \qquad d = \lceil \ln(1/\delta) \rceil$$

Heavy hitters are estimated in fixed memory with d hash rows of w counters: the estimate never underflows and overshoots by at most εN with probability 1 − δ. Unknown floods are shed before any per-source table is touched.

Bounded by design

Reflection is bounded, not just claimed

$$R_{\text{refl}} \leq \frac{1}{k}$$

a truncated (TC=1) reply slips out only every k-th drop, so the reflection ratio can never exceed 1/k.

Proof, not prefix

$$\text{cookie} = \text{SipHash}_K(\text{src} \,\|\, \text{dst} \,\|\, t)$$

a keyed MAC (SipHash-2-4) or a completed TCP exchange promotes a source to tier T2 — unforgeable off-path.

Verifier-proven parse

fixed loop bounds and a per-CPU scratch buffer — the kernel's eBPF verifier statically rejects any unbounded walk before load.

The results

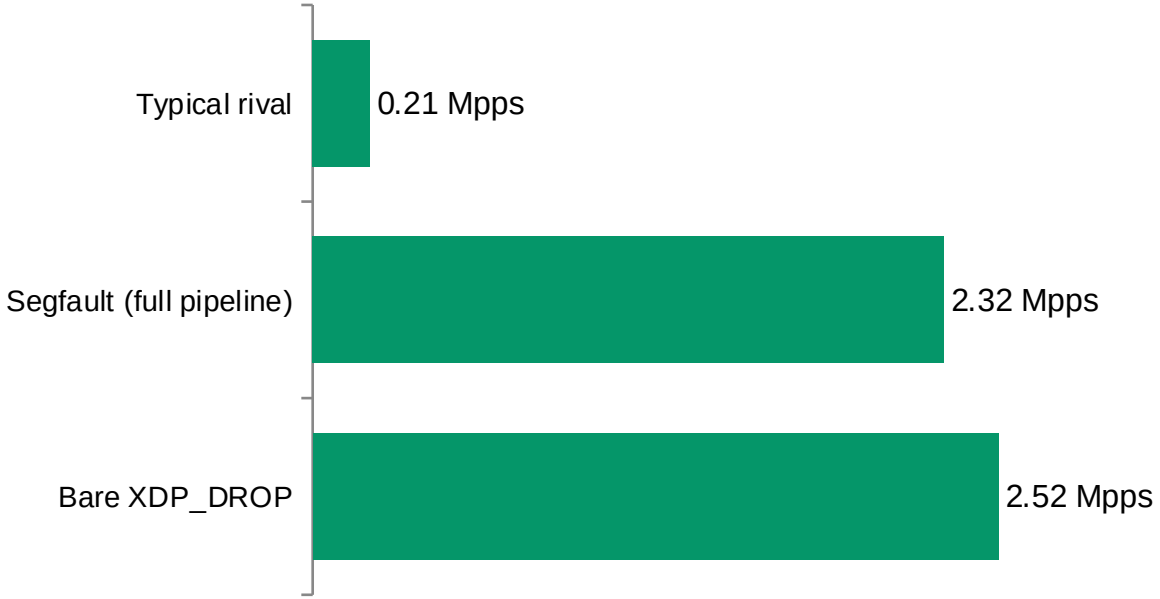
Scenario	Best baseline	Segfault
distributed	119 / 544 ms	3719 / 1.1 ms
adaptive	65 / 432 ms	3670 / 1.1 ms
prefix-spoof	0	3729 / 0.4 ms
water-torture	68 / 680 ms	3684 / 1.1 ms

queries completed / p99 tail latency

Performance

Throughput and CPU cost, measured in-kernel (veth + pktgen, no virtual switch).

In-kernel drop rate



Server-hardware ceiling

Server NIC	Line-rate cap	Cores
10 GbE	11.6 Mpps	~1–2
25 GbE	29 Mpps	~3
100 GbE	116 Mpps	~12

c ≈ 290 cycles / packet → ~6.9–10 Mpps per core; 100 GbE needs ≈ 12 cores.

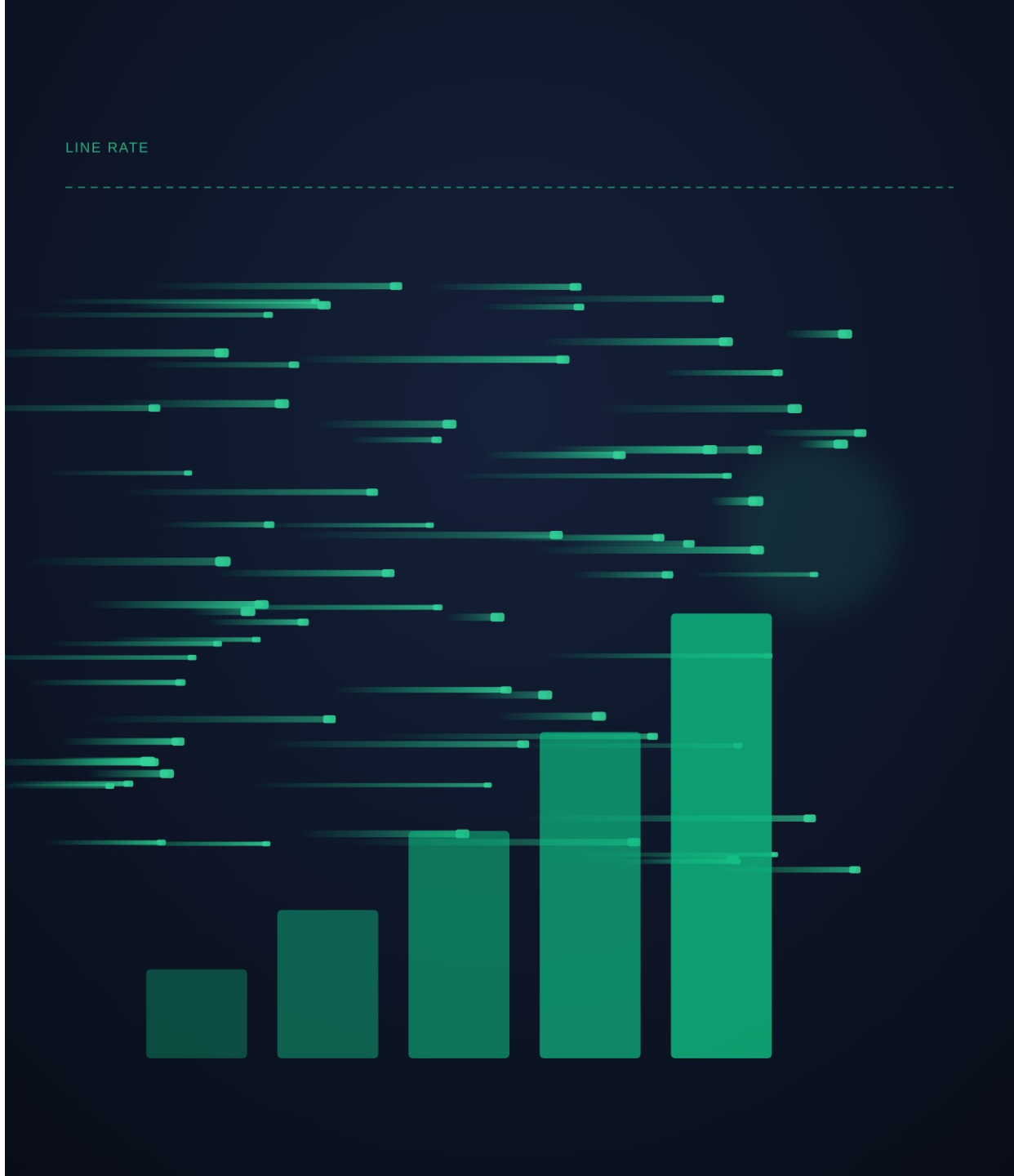
Segfault sheds 2.32 Mpps at 21% of one core — only ~40% more CPU per packet than a do-nothing XDP drop (2.52 Mpps at 16%), and ~10× the rate of a typical rival. Neither was CPU-saturated, so the true ceiling is higher.

~40% over a bare XDP drop in-kernel; line-rate-bound on server NICs.

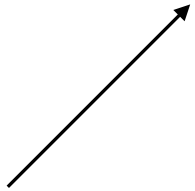
$$C = \min\left(R_{\text{line}},\quad N_{\text{core}} \cdot \frac{f}{c}\right)$$

c ≈ 290 cycles / packet measured → 6.9–10 Mpps per core; a 100 GbE line rate needs ≈ 12 cores.

Built to run at line rate.



“Segfault asks who,
and what it costs.”



Segfault design principle



Thank you

Built by Team Segfault

Debangan Mali · Saraswata Chatterjee · Partha Chakraborty (Faculty)

debanganmali99@gmail.com · IEM Kolkata · A3-PS022-TC005